Trabajo Práctico N.º 8 · Food Store en la feria

La base de la tablet y el cierre de la capa de datos

Asignatura	Base de Datos II · Tecnicatura Universitaria en Programación · UTN
Integrantes	Pamela Chirino · Lucas Agüero · Mayra Mule
Herramientas	PostgreSQL 16 · SQLite 3 · Kiro · OpenCode · Claude
Ciclo lectivo	2026 · Unidad 8 · semanas 12 y 13

Regla de oro: el móvil propone, el servidor dispone. La tablet solo consulta y arma borradores; precio, stock y ventas los decide PostgreSQL. **SIMULADO_APROBADO no es una venta.** Todo se ejecutó sobre una copia de pruebas con datos inventados; las salidas completas están en `docs/evidencia/`.

1.a · Qué datos viajan a la tablet

Viajan (v_snapshot_movil)	Para qué los usa la tablet	NO viajan	Por qué no
id	Identificar el producto igual que en el servidor	usuario: email, password_hash, rol	Datos personales y credenciales: no se necesitan para vender y una tablet se puede perder
nombre	Mostrar el catálogo	pedido y detalle_pedido históricos	Son ventas confirmadas: las maneja solo el servidor
precio_centavos	Calcular el total y guardar el precio visto	descripcion, imagen_url	No hacen falta para armar el pedido (ahorra espacio)
stock (como stock_visto)	Orientar al vendedor, no decidir	categoria_id, actualizado_en	No se usan en la feria
disponible, eliminado (lápida)	Ocultar lo que no se vende sin borrar la fila		
fecha_snapshot	Saber de cuándo es la copia		

Solo viajan datos de producto, porque es lo único que la tablet necesita para mostrar el catálogo y armar borradores. Los datos de usuarios y las ventas se quedan en el servidor por privacidad y porque no le corresponden a la tablet. **La fuente de verdad es PostgreSQL**: lo que hay en la tablet es una copia con fecha que puede estar vieja; por eso el semáforo vuelve a comparar contra el servidor antes de cualquier cosa.

2.a y 2.b · Instantánea y semáforo

La vista `v_snapshot_movil` convierte el precio a centavos e incluye los productos dados de baja como lápidas. Se exportó con `\copy … CSV HEADER`, se importó en `snapshot_servidor` y se pasó a `producto_cache` con un upsert (`cargar_snapshot.sql`). **Fecha de la copia inicial: 2026-11-09 18:26:37** (8 productos). El semáforo (`semaforo.sql`) agrupa por solicitud, cuenta los ítems con problema usando `LEFT JOIN` contra la instantánea y solo actualiza filas **PENDIENTE**; además deja un motivo legible por ítem.

```
ROJO     RECHAZADO          producto inexistente / no disponible / dado de baja / cantidad > stock
AMARILLO CONFLICTO          sin rojo, pero precio_centavos_visto <> precio_centavos
VERDE    SIMULADO_APROBADO  todo coincide (no es una venta)
```

2.c · Los casos

Preparación: con la instantánea 1 se armaron los borradores de C2 a C5 (`casos/01_borradores_casos.sql`). Después, como administradores, se cambió la copia del servidor (`servidor/casos_servidor.sql`): la hamburguesa pasó de $5500 a $6000 y el choripán se dio de baja lógica. Se exportó la instantánea 2, se cargó y se corrió el semáforo.

Caso	Comando	Resultado obtenido	Explicación
C2	semaforo.sql	CONFLICTO · producto 1 cambió de precio (550000 -> 600000 cent.)	El precio visto en la feria ya no es el del servidor. No hay rojo, entonces es amarillo.
C2 acepta	casos/03_c2_cliente_acepta.sql + semaforo.sql	solicitudes_evaluadas_ahora = 1 · nueva solicitud …02c2 SIMULADO_APROBADO	El cliente acepta: se actualiza el precio del borrador y se envía con UUID nuevo. La vieja queda en CONFLICTO como historial.
C3	semaforo.sql	RECHAZADO · producto 4 sin stock (pide 50, hay 40)	Se piden más empanadas de las que hay según el servidor. No se toca el stock.
C4	semaforo.sql + casos/05_c4_catalogo.sql	RECHAZADO · producto 5 dado de baja. El catálogo ya no muestra el choripán, pero su fila sigue con eliminado = 1	La lápida avisa la baja sin borrar la fila, así el borrador viejo sigue apuntando a algo válido.
C5	semaforo.sql	SIMULADO_APROBADO · coincide con la instantánea	Precio, stock y estado iguales. Es solo una simulación: el servidor todavía tiene que confirmar.

Caso	Comando	Resultado obtenido	Explicación
C6	casos/04_c6_reenvio.sql + semaforo.sql	UNIQUE constraint failed: cola_sync.solicitud_uuid · solicitudes antes 6 / después 6 · semáforo: 0 evaluadas	El UUID repetido choca con la clave primaria y, como solo se evalúan PENDIENTE, repetir no cambia nada.

C6 en un sistema de verdad (parada 8): la protección real va en el servidor. PostgreSQL guarda el `solicitud_uuid` en una tabla de solicitudes procesadas con restricción UNIQUE, dentro de la misma transacción que crea el pedido; si la tablet reintenta por falta de señal, el servidor devuelve la respuesta ya dada en vez de duplicar la venta.

Control en PostgreSQL: pedidos y stock no cambiaron

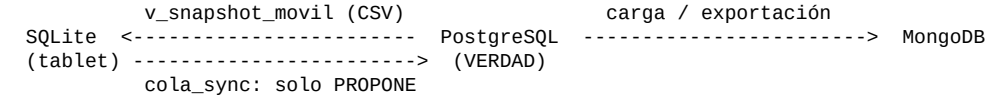
```
SELECT (SELECT count(*) FROM pedido) AS cant_pedidos, (SELECT count(*) FROM detalle_pedido) AS cant_detalles,
       (SELECT sum(stock) FROM producto) AS stock_total,
       (SELECT md5(string_agg(id||':'||stock, ',' ORDER BY id)) FROM producto) AS huella_stock;

           cant_pedidos | cant_detalles | stock_total | huella_stock
ANTES:          2       |       5       |     190     | aec8e1a84aeb17ff5a1c7f1f9b251203
DESPUÉS:        2       |       5       |     190     | aec8e1a84aeb17ff5a1c7f1f9b251203
```

La huella compara el stock producto por producto, no solo la suma. Los únicos cambios en el servidor (precio de C2 y baja de C4) los hizo el administrador para preparar los casos, nunca la tablet.

3.a · Mapa de la capa de datos

Motor	Qué guarda	Quién manda	Cómo se relaciona
PostgreSQL (servidor)	Usuarios, categorías, productos (precio, stock, disponible, baja), pedidos y detalles	**Manda sobre todo lo transaccional:** precio, stock, ventas y usuarios	Es la fuente de verdad. Publica v_snapshot_movil para la tablet y alimenta a MongoDB
MongoDB (Unidad 7)	Documentos de la Unidad 7 armados a partir de PostgreSQL (datos de lectura y consulta flexible)	Manda solo sobre los datos propios de esa colección; los datos de producto/pedido son copia	Se carga desde PostgreSQL; si hay diferencia, gana PostgreSQL
SQLite (tablet)	producto_cache (copia con fecha), borradores, ítems y cola_sync	Manda solo sobre sus borradores y la cola de solicitudes	Recibe instantáneas; envía solicitudes que el servidor puede rechazar



3.b · Plan de mantenimiento (agregado al plan de la Unidad 6)

Pregunta	Respuesta
¿Qué se respalda y cómo se prueba la restauración?	PostgreSQL completo con `pg_dump -Fc` todas las noches (no tablas sueltas: probamos que con `-t producto` la restauración falla y no vuelve la vista). Una vez por semana se restaura en una base aparte (`pg_restore`) y se verifica con `control_servidor.sql` y `SELECT count(*) FROM v_snapshot_movil`. De la tablet se copia el archivo .db al cerrar la feria (solo borradores y cola).
Si se restaura un respaldo viejo de PostgreSQL, ¿qué hacen las tablets?	Su catálogo puede ser más nuevo que el servidor. Se descarta producto_cache, se baja una instantánea nueva y se vuelven a evaluar las PENDIENTE. Las que estaban en SIMULADO_APROBADO pero no llegaron a confirmarse se vuelven a enviar con UUID nuevo, porque el servidor restaurado no las conoce. Nada se da por vendido.
¿Cada cuánto y quién revisa CONFLICTO y RECHAZADO?	Al cierre de cada jornada de feria, el encargado del food truck (rol ADMIN). Los CONFLICTO se resuelven con el cliente (acepta el precio o se cancela); los RECHAZADO se anotan con su motivo. Si un motivo se repite (por ej. falta de stock), se revisa la carga de stock antes de la próxima feria.

3.c · Code review con IA

Se pidió a la IA, en modo revisión (solo leer y proponer, sin modificar), que revisara los scripts del servidor y de la tablet. Cada hallazgo se comprobó en la base; las salidas están en `docs/evidencia/verificaciones_code_review.txt`.

#	Unidad / script	Hallazgo	Decisión	Motivo y cómo se comprobó
H1	U8 · todos los .sql de movil/	PRAGMA foreign_keys se apaga en cada conexión nueva	**Aceptado**	Conexión nueva: PRAGMA da 0 y un ítem con producto 999 se guarda. Con el PRAGMA en la primera línea da FOREIGN KEY constraint failed. Se dejó en todos los scripts y en el README.
H2	U8 · semaforo.sql	Con LEFT JOIN, un producto ausente de la instantánea da NULL y SUM no lo cuenta	**Corregido**	En una copia se borró el producto 2 de snapshot_servidor: la versión de la pista daba SIMULADO_APROBADO; la corregida (s.id IS NULL y CASE ... THEN 1 ELSE 0) da RECHAZADO.

#	Unidad / script	Hallazgo	Decisión	Motivo y cómo se comprobó
H3	U8 · schema_sqlite.sql	Precio en REAL sería más simple	**Descartado**	En SQLite 0.1 + 0.2 = 0.3 da 0 (falso); con enteros 10 + 20 = 30 da 1. Los centavos evitan errores de redondeo al sumar totales.
H4	U1 · vista del servidor	(precio * 100)::INTEGER podría truncar centavos	**Descartado**	precio es NUMERIC(10,2), exacto: 5499.99 da 549999 y 0.07 da 7. Un valor con 3 decimales ya se redondea al guardarse (1.005 queda 1.01).
H5	U8 · producto_cache	Trigger que descuente stock_visto al agregar un ítem	**Descartado**	Rompe la regla de oro: la tablet no decide stock. grep confirma que ningún script de movil/ hace UPDATE de stock y la huella de stock en PostgreSQL no cambió.
H6	U6 · plan de respaldo	Respaldar solo la tabla producto alcanza para la tablet	**Descartado** / aceptado el respaldo completo	pg_restore de un dump con -t producto falla (falta categoria) y no vuelve v_snapshot_movil. El respaldo completo -Fc restaura y la vista devuelve 8 productos.